INFORME TÉCNICO · CATÁLOGO DE CONFIGURACIONES

Configuraciones y archivos extra para subir el t/s

Catálogo de vías para acelerar Qwen3.6-35B-A3B APEX-I-MINI en el rig: flags del servidor (colocación de tensores, concurrencia, prefill), archivos extra (requant por tensor, metadatos del GGUF, binarios) y Windows. Especulación MTP excluida; con evidencia, ganancia esperada y antivenenos medidos.

LAIA · Proyecto Qwen APEX

Preparado para el Director — catálogo sin plan de ejecución

26 OCT 2026 · I3-10100F · GTX 1650 4 GB · 16 GB DDR4-2666 · LLAMA.CPP B11856

Índice

1. Resumen ejecutivo 1

2. Física del rig y reglas del catálogo 2

3. Memoria de veredictos: lo ya probado 3

4. Eje A — Colocación por tensor: -ot/--override-tensor 4

5. Eje B — Concurrencia y carga: -np/--parallel 5

6. Eje C — Prefill: op-offload, batches y prioridad 6

7. Eje D — Archivos extra: binarios, GGUF y metadatos 7

8. Eje E — Windows: energía, Defender y pagefile 9

9. Apéndice: flags verificadas del build b11856 9

10. Antivenenos: lo que NO repetir 10

11. Matriz maestra y fuentes 11

1. Resumen ejecutivo

Este informe es el catálogo de configuraciones de llama.cpp y de archivos extra capaces de subir los tokens por segundo del rig (i3-10100F, GTX 1650 4 GB, 16 GB DDR4-2666) con el modelo Qwen3.6-35B-A3B APEX-I-MINI de 14.27 GB. El punto de partida medido es una ancla de 21.53 t/s de decode [M] (ANCHOR-B, protocolo de 4 pasadas x 256 tokens del barrido-30), un campeón de colocación de 22.49 t/s [M] (NCM37C4, con -ncmoe 37 y KV q4_0) y un campeón de prefill de 57.56 t/s [M] (ORO-UB1K, +26.5% sobre la base), ya aplicado en el lanzador v2.1. La meta macro del proyecto sigue siendo duplicar el t/s, con umbral mínimo de +50% (32.3 a 33.6 t/s) e ideal de 43 a 45 t/s.

El alcance lo fijó el Director antes de empezar: formato PDF largo de 15+ páginas, cobertura total (flags del servidor, archivos extra y nivel Windows), sacrificio de calidad moderado, especulación excluida (el MTP y los drafts ya están investigados y en manos del tester-mtp con el protocolo T2) y formato de catálogo puro, sin plan de ejecución: cada vía se documenta con su evidencia, su ganancia esperada, su riesgo y la manera de medirla, y el orden de ataque lo decide el Director. El entregable se sube además al repositorio del proyecto en una carpeta nueva, como pidió el Director.

Lo genuinamente nuevo respecto a los barridos-25 y -30 son dos grupos. Primero, cinco vías de configuración nunca tocadas en el rig: la concurrencia por slots (-np con batching continuo y sampling en GPU), la colocación quirúrgica de tensores (-ot/--override-tensor, distinta de la colocación por capas -ncmoe ya explorada), el umbral de offload de prefill (variable GGML_OP_OFFLOAD_MIN_BATCH), la prioridad del hilo de prefill (--prio-batch) y la escalada de batches por encima de UB1K. Segundo, las vías de archivo: el binario nuevo que desbloquea el -fit abortado, el requant moderado por tensor con --tensor-type e --imatrix, la cirugía de metadatos del GGUF para el enigma de expertos activos, y los binarios alternativos (Vulkan, ik_llama.cpp) con su gate.

La regla de oro del proyecto no cambia: nada se adopta sin medir en el rig, y toda ganancia marcada [T] o [D] en este catálogo es una hipótesis con física detrás, no una promesa. El capítulo 9 recopila los antivenenos: palancas ya medidas que perdieron, con su número y su mecanismo, para que ninguna hora del rig vuelva a gastarse en un experimento que ya tiene veredicto. El capítulo 11 cierra con la matriz maestra del catálogo y las fuentes verificadas con cita verbatim.

21.53 a 22.49 t/s	x1.5 a 2.5	8 + 5
ancla y campeón de decode medidos [M] en el barrido-30	t/s agregado teórico [T] de la concurrencia -np 2-4 en CPU-MoE	vías nuevas de configuración y de archivo catalogadas

Vía	Eje	Tipo	Ganancia esperada	Estado
Concurrencia -np 2-4 (+ -cb, -bs)	B	Config	x1.5-2.5 agregado [T]	Nunca medida
Requant IQ3 por tensor (+imatrix)	D	Archivo	+15-30% decode [T]	Nunca medida
Metadatos expert_used_count	D	Archivo	+15-28% condicional [T]	Gate pendiente
-fit real en binario nuevo	D	Binario	+5-8% [T]	Abortado por bug #27501
GGML_OP_OFFLOAD_MIN_BATCH	C	Entorno	Prefill ±10-30% [D]	Nunca medida
-ot fino + KV q4_0 + c 4096	A	Config	+0.3-1.5% [T]	Variante no probada
powercfg Alto rendimiento	E	Windows	0-5% [T]	Nunca medida

Tabla 1. Las siete vías mayores del catálogo. Ninguna se adopta sin medir.

2. Física del rig y reglas del catálogo

El techo del decode de este rig es un techo de ancho de banda. La DDR4-2666 en doble canal ofrece 42.6 GB/s teóricos, de los que el proyecto deriva 26 a 28 GB/s efectivos [D]. Con los expertos en CPU (-cmoe), el modelo lee unos 0.95 a 1.05 GB por token solo de expertos (10.8 GB de pesos MoE activos a ~3.3 bpw [D]), a los que se suma la lectura de atención y KV: 1.2 a 1.3 GB por token en total [D]. Ese presupuesto da un techo de 21 a 22 t/s [T] que la medición confirma casi exactamente (21.5 t/s [M]). Cualquier vía que no juegue con esas variables está peleando contra la física, no contra la configuración.

El cuello es además mixto: a la banda de la DDR4 se suma el cómputo de dequantización de los cuants Q3 en cuatro núcleos AVX2, con una eficiencia estimada de 25 a 30% [D]. De ahí salen las tres palancas legítimas que ordenan todo este catálogo: reducir los bytes leídos por token (requant por tensor, metadatos de expertos activos), añadir cómputo gratis (colocar tensores en la VRAM que sobra, dejar que el -fit empaquete solo) y amortizar o reusar lecturas (batching por slots, caches de prompt y de slot). Las vías de Windows entran como cuarta familia porque estabilizan relojes y E/S, es decir, hacen que las otras tres se midan limpias.

Las etiquetas del proyecto mantienen su contrato. [M] significa medido en el rig con el protocolo del barrido (4 pasadas de 256 tokens con cache_prompt y temp 0, descarte de la primera, pausas térmicas, ruido declarado de ±0.3 a 0.5 t/s). [T] significa teoría física aplicada a este hardware. [F] significa fuente externa verificada con cita verbatim, listada en el capítulo 11. [D] significa derivación o estimación propia del proyecto a partir de datos medidos. Las herramientas de medida existentes son BARRIDO-QWEN-30.bat (37 slots de configuración), el tester-mtp.py con su modo --protocolo y llama-bench.

Dos presupuestos acotan qué se puede pedir. En VRAM: los 4 GB de la GTX 1650 ya cargan atención, embeddings y KV q8_0 (unos 3.1 a 3.4 GB [D]; la KV de este modelo es chica, ~0.34 GB a 16k, por ser 30/40 capas DeltaNet [D]), y cada experimento que pidió más acabó en castigo medido (FIT-C 13.25 t/s [M], familia -ncmoe con -ngl fijo entre 5.58 y 7.33 [M]). En RAM: 14.27 GB de modelo sobre 16 GB instalados convierten los pageouts en la segunda causa documentada del techo [D]. Toda vía de este catálogo declara cuánto pide de cada presupuesto antes de medirse, y el apéndice del capítulo 9 fija qué flag exacta soporta cada vía, verbatim del build

b11856 que corre en el rig.

3. Memoria de veredictos: lo ya probado

El catálogo se apoya en una base empírica de 62 configuraciones medidas entre el barrido-25 (25 configs, con veredictos en aprendido.md) y el barrido-30 (37 configs, CSV con decode, prefill frío y acceptance de ngram). Esta tabla resume los veredictos que condicionan el catálogo: los campeones que el lanzador ya usa, los neutros que no merecen más minutos y los venenos que el capítulo 10 convierte en reglas. La lección transversal de los dos barridos es doble: las palancas ganadoras no se suman entre sí (los tres combos dinámicos del barrido-30 perdieron), y el techo a coste cero sin tocar BIOS está en torno a 21.5 a 22 t/s [M].

Config (id barrido-30)	Qué probaba	t/s [M]	Veredicto
ANCHOR-B	base con -lm none	21.53	Ancla del proyecto
NCM37C4	-ncmoe 37, -c 4096, KV q4_0	22.49	Campeón decode; pendiente de combinar con UB1K
FIT-A	-ngl auto, -ncmoe 34, -fitt 256	22.40	Campeón nominal; el fit ABORTO (bug #27501): no es "con fit"
ORO-UB1K	-ub 1024, -b 4096	21.55 (prefill 57.56)	Ganador por coste total; aplicado en el lanzador v2.1
ORO-TB8	-tb 8 (hilos de prefill)	21.45	Neutro
PRIO0/PRIO3, POLL20/POLL80	prioridad y polling	20.69 a 21.47	Neutros
EXP4 a EXP10	override-kv expert_used_count	21.32 a 21.54	Nunca aplicó: dosis-respuesta nula y control EXP10 plano
ORO-NGRAMS / ORO-NGRAMM	ngram-simple / ngram-mod	18.09 / 20.70	Apagar: acceptance 0.0% y 15.6% (<20%)
MTP (barrido-25, 8 variantes)	draft-mtp, n-max 1-3, p-min 0.10	16.62 a 18.53	Negativo; retest T2 pendiente con acceptance
CR03/CR14, CMASK55, STRICT*	afinidad y máscaras de CPU	13.37 a 19.48	Veneno
-t 3 / -t 5	hilos de generación	18.29 / 18.75	Pierden; -t 4 es el punto dulce (el HT estorba)
moe-cache-mib 256 / 512	caché GPU de expertos	13.51 / 17.22	Veneno con 4 GB: desborde a RAM compartida
-ncmoe 16 a 40 con -ngl 99 fijo	expertos a GPU por capas	5.58 a 7.33	Veneno: VRAM desbordada; solo con -ngl auto
KV q4_0 sola, -c 8192, --no-op-offload, REPACKOFF	ajustes "liberadores"	20.21 a 20.48	Pierden ~1 t/s
COMBO-TOP/EXP/X3, COMBO-A	ganadores apilados	18.98 a 21.33	Los ganadores no componen

Config (id barrido-30)	Qué probaba	t/s [M]	Veredicto
LMDIO, NOHOST, NOWARM, CRAM512, SHIFT	carga y cachés varias	21.38 a 21.57	Neutros

Tabla 2. Veredictos del barrido-30 (protocolo 4x256) y del barrido-25. Base común del catálogo.

Dos matices de esta memoria condicionan directamente los capítulos siguientes. El primero: FIT-A midió 22.40 t/s [M] pero su log muestra el ajuste abortado ("failed to fit params ... tensor_buft_overrides already set by user, abort", consistente con el issue #27501), así que el número corresponde a la config sin fit real; el retest con un binario nuevo es la vía D1. El segundo: la familia EXP midió plano porque el override de metadatos nunca llegó al modelo; la vía de archivo D3 retoma ese enigma tocando el GGUF en vez del comando.

4. Eje A — Colocación por tensor: -ot/--override-tensor

El build b11856 del rig trae la palanca de colocación más fina de llama.cpp, verificada en el dump del servidor [F]: -ot, --override-tensor, que asigna a cada tensor que cumpla un patrón de nombre (expresión regular) un tipo de buffer concreto (CPU0, CUDA0, etc.), con lista separada por comas y su gemelo -otd para el modelo draft. Donde -cmoe es un interruptor global y -ncmoe N cuenta capas, -ot es cirugía: decide tensor a tensor qué se queda en GPU y qué viaja a la RAM, y es la herramienta estándar de la comunidad para MoE en GPUs pequeñas.

La receta canónica viene del PR #11397 de llama.cpp [F], verbatim: "for example, to force the experts in MoE models to stay on the CPU, while offloading the rest to the GPU, you could use -ngl 99 -ot exps=CPU". La guía de MoE offload de HuggingFace (enero de 2026) [F] usa patrones por rango de capas como -ot "blk\.([0-9]|[1-2][0-9]|30)\.=CUDA0,exps=CPU" (que traduce como "assign all routed expert FFN to the CPU"), y el patrón popular de r/LocalLLaMA [F] para no descargar capas sino tensores es -ot ".ffn_.*_exps.=CPU". En este rig esos patrones son directamente aplicables porque el modelo usa la nomen-clatura ffn_*_exps verificada en los logs del barrido (blk.40 incluido).

La relación con lo ya probado es la clave del capítulo. El barrido-30 exploró el eje por capas (-ncmoe 37 dio 22.49 t/s [M]: tres capas de expertos a GPU más KV q4_0 y contexto 4096) y el eje global (-cmoe, la base del lanzador), pero no el eje por tensor. Variantes sin probar con -ot: dejar en GPU solo los down_exps de las últimas capas, repartir gate/up frente a down, o mover a GPU únicamente las capas cuyo hueco de VRAM lo permita sin tocar -ngl. Como palanca sola la expectativa es modesta, +0.3 a 1.5% [T]; como habilitadora es grande, porque tras un requant (capítulo 7, D2) sobra VRAM y -ot es quien la reparte.

Riesgo, unified memory y guardarráiles

El castigo por pasarse de VRAM ya está medido: FIT-C se fue a 13.25 t/s [M] y la familia -ncmoe con -ngl 99 fijo se hundió entre 5.58 y 7.33 [M] porque el auto-fit no podía actuar. Regla del eje: con expertos a GPU parcialmente, solo -ngl auto con -fitt, nunca -ngl 99 fijo, y verificación de VRAM libre con nvidia-smi antes y después. La vía hermana GGML_CUDA_ENABLE_UNIFIED_MEMORY=1 (PR #8035 [F]: "useful when the model barely fits in VRAM and inference causes OOM") queda aquí como no recomendada [T]: el paging por PCIe del driver es el mismo mecanismo que hundió la caché moe-cache (capítulo 10), y la discusión #14269 documenta el crecimiento anómalo de RAM host. Solo como experimento de última instancia y sabiendo que el decode puede desplomarse.

Cómo se mide

Dos slots de barrido bastan. A-OT1, control de equivalencia: base del lanzador más -ot "exps=CPU", que debe clavar la ancla (si no clava, el patrón no aplicó y hay que depurar el regex). A-OT2, la apuesta: base con -c 4096 y KV q4_0 más un -ot fino que deje 2 a 3 capas de expertos en GPU por nombre de tensor, comparado contra NCM37C4 (22.49). Criterio de adopción pre-registrado del proyecto: superar el campeón por más de +0.5 t/s en el protocolo de 4 pasadas. El coste de medición es de unos 12 minutos por config con el barrido estándar.

5. Eje B — Concurrencia y carga: -np/--parallel

La vía más grande jamás probada en el rig no toca los pesos ni la colocación: reparte el trabajo. El dump b11856 trae -np, --parallel N (number of server slots, por defecto -1 = auto) [F], el batching continuo -cb activo por omisión, el sampling en GPU -bs, --backend-sampling (verbatim del dump: "enable backend sampling (experimental) (default: disabled)") [F], el presupuesto de caché -cram (default 8192 MiB) [F] y la similitud de prompt por slot -sps (default 0.10) [F]. Ninguna medición del proyecto ha corrido nunca con más de un slot: todo el barrido es monostream.

La evidencia externa es fuerte. En la discusión #18308 de llama.cpp (enero de 2026) [F], el usuario aheid publica la tabla de llama-batched-bench de su gpt-oss-20B en GPU: la generación agregada pasa de 295.25 t/s con 1 secuencia a 1430.11 t/s con 32 (x4.8), con "I'm not really getting a significant speedup of -np beyond 4". El mantenidor ggerganov precisa el matiz que importa aquí [F], verbatim: "The llama-batched-bench does not measure the time for the sampling — it's done on the CPU and when running the real llama-server it makes a difference, especially for big parallelization cases (i.e. we need to sample for each parallel sequence)", y recomienda para servidores saturados el PR #17004 con CUB 3.2 más --backend-sampling, avisando: "if you don't saturate it, then you will likely not see an improvement".

La física en CPU-MoE juega a favor [T]. En decode por lotes, los 10.8 GB de expertos se leen de la DDR4 UNA vez por paso y sirven a todas las secuencias del lote: el coste de banda se reparte entre N conversaciones en vez de multiplicarse, y el agregado escala hasta que el cómputo de dequant en los 4 núcleos satura. Estimación honesta para este rig: -np 2 a 4 daría un agregado de x1.5 a 2.5 sobre la ancla [T], con caída de 30 a 50% en el t/s de cada chat individual [T]. Es la vía que alimenta directamente el objetivo de carga del proyecto (concurrencia), no la latencia de una sola conversación.

Costes, límites y guardarráiles

Tres límites concretos. KV por slot: la KV de este modelo es chica por las 30/40 capas DeltaNet (~0.34 GB a 16k [D]), pero se multiplica por N: con -np 4 a 16k son ~1.4 GB de VRAM extra [D], así que conviene medir con -c 4096 (el campeón NCM37C4 ya vive con ese contexto) y, si desborda, limitar por slot con --kv-unified-per-slot [F dump]. Muestreo: con N slots el sampling en CPU compite con el decode por los 4 núcleos, y -bs lo mueve a GPU [F dump], con la condición de saturación del servidor que fija ggerganov [F]. RAM: 14.27 GB de modelo dejan el margen más fino del rig; el pagefile ampliado del eje E es seguro de vida para esta vía.

Cómo se mide

El tester-mtp actual es monostream, así que esta vía exige su propio mini-instrumento: un driver de N clientes HTTP concurrentes (hilos de la librería estándar contra /v1/chat/completions) que mida t/s agregado, t/s por cliente y TTFT. Diseño de medición: -np 2, 3 y 4 con -c 4096 sobre la base del lanzador, más una rama con -bs activo; guardarrail de RAM antes de cada corrida. El criterio de éxito es de catálogo: agregado superior a x1.5 sin colapso de latencia individual. Coste estimado: 30 a 45 minutos entre script y corridas.

6. Eje C — Prefill: op-offload, batches y prioridad

El prefill del rig ya dio su salto grande (57.56 t/s [M] con -ub 1024 -b 4096, +26.5% sobre la base de 45.50), y ese ganador está aplicado en el lanzador v2.1. El eje explora lo que quedó sin escalada: UB512 había medido peor (20.77 [M] en el barrido-25), pero -ub 2048 con -b 8192 nunca se midió; la expectativa es de +5 a 12% de prefill [T] con el riesgo conocido de buffers de VRAM (el castigo FIT-C 13.25 [M] marca la frontera). El decode no se toca en este eje: es puro tiempo-hasta-la-primera-palabra.

La pieza nueva con fundamento externo es el umbral de offload de prefill. La guía de MoE offload de HuggingFace lo documenta verbatim [F]: "(llama.cpp main only) The environment variable GGML_OP_OFFLOAD_MIN_BATCH can be used to override the default threshold of 32 for triggering offloading prompt processing fully to the GPU. If you have low PCIe bandwidth to your GPU, or you have a very large amount of model weights on CPU, 32 is probably too low... Your prompt batch likely needs to be at least several hundred for this to be "worth" doing." En el rig, -cmoe deja ~10.8 GB de expertos en CPU [D]: con ubatch de 1024, cada lote de prefill dispara copias de pesos por el PCIe 3.0 de la GTX 1650 (~12 a 14 GB/s reales [D]).

La lectura de este rig tiene un dato que ancla la hipótesis: apagar el offload por completo (--no-op-offload) midió 20.48 t/s [M], peor que la ancla, es decir, el offload de operaciones AYUDA hoy. El eje propone afinar el umbral, no matarlo: tres valores del entorno (0 para apagar solo el offload de prefill masivo, el default 32 como control, y 4096 para que solo los lotes enormes lo activen) separan las dos hipótesis posibles, que el PCIe está cobrando peaje o que el offload ya está amortizado. Es una medición de 20 minutos con el prefill frío de 293 tokens del barrido más un prompt largo de 4k para ver la escala.

--prio-batch: la prioridad que falta

El dump trae la palanca hermana de la prioridad ya probada [F]: --prio-batch N con valores "0-normal, 1-medium, 2-high, 3-realtime", aplicable también como --prio-batch-draft. La familia PRIO del barrido-30 midió neutra para decode (21.09 a 21.47 [M]), pero la prioridad del hilo de batch/prefill nunca se probó. Expectativa pequeña [T]: estabiliza el prefill cuando el escritorio está vivo, que es exactamente el escenario de uso del Director. Coste de medición: una config del barrido, 12 minutos.

7. Eje D — Archivos extra: binarios, GGUF y metadatos

Aquí la palanca es un archivo: el binario del servidor, un GGUF requantizado, los metadatos del GGUF o los binarios de un backend alternativo. Son las vías con mayor techo del catálogo porque atacan directamente los bytes por token o el empaquetado de VRAM, y también las más caras de preparar (descargas, cuantizados, scripts de metadatos). Cada sub-vía trae su gate: la condición verificable que decide si merece minutos de rig.

D1 — Binario nuevo y el retest del -fit abortado

La cadencia de llama.cpp es de aproximadamente un release diario [F]: el histórico de releases registra b11871 (25-oct-2026, dos días más nuevo que el b11856 del rig), b11747 (19-oct, "Fixes CUDA flash-attention race conditions") y b11640 (15-oct, "Vulkan backend gets L2-friendly workgroup scheduling for matrix multiplication"). El zip oficial ya verificado para el rig es llama-bXXXX-bin-win-cuda-12.4-x64.zip más sus DLLs cudart-llama-bin-win-cuda-12.4-x64.zip [F]. El premio concreto del eje: FIT-A midió 22.40 t/s [M] con el ajuste abortado (log verbatim: "failed to fit params ... tensor_buft_overrides already set by user, abort", consistente con el issue #27501); si el build nuevo trae el fix, el -fit REAL empaqueta capas de expertos en la VRAM libre por sí solo, con expectativa de +5 a 8% [T] sobre el 22.4. Es la vía con mejor relación ganancia-minutos del eje.

D2 — Requant moderado por tensor (--tensor-type e --imatrix)

La herramienta llama-quantize admite cuantización selectiva por tensor [F], verbatim de la manpage: es "an advanced option to selectively quantize tensors", y "--imatrix uses data in file generated by llama-imatrix as importance matrix". La discusión #12741 (nacida del PR #12511) documenta el flujo "Experimenting with custom quants using --tensor-type" [F], y el estudio de bartowski sobre mapas de layout por tensor (septiembre de 2026) confirma la asimetría que el eje explota: los expertos toleran menos bits que la atención, mientras que los shexp "were pretty sensitive despite being incredibly small" [F]. La receta moderada acordada: atención, shexp y embeddings intocados; solo los ffn_*_exps un escalón abajo (IQ3_XXS o IQ3_XS con imatrix).

La matemática del eje [D]: un GGUF de ~12.5 a 13 GB reduce los bytes por token de 1.2-1.3 a ~1.0-1.1 GB, con t/s esperado de 24 a 27 [T], sin contar el hueco de VRAM que sobra para -ot (capítulo 4). El coste de calidad es el gate: la familia IQ mueve la perplexity entre +17 y +37% según esquema y imatrix [F, arXiv 2601.14277], así que el protocolo exige validar chateando antes de adoptar. Dos avisos honestos: el informe de 20 vías ya halló que el escalón útil de este modelo es la familia IQ2 (el IQ3_K_M puro engorda el GGUF, de ahí que aquí se hable de

IQ3_XXS/IQ3_XS por tensor y no de requant global), y los quants exóticos pueden cobrar en dequant por el cuello mixto [D], así que la medición manda sobre la teoría. Los comandos de generación (llama-imatrix sobre texto y llama-quantize con --imatrix más --tensor-type para los exps) quedan documentados en la matriz final.

D3 — Cirugía de metadatos: el enigma EXP

La familia EXP del barrido-30 (--override-kv sobre expert_used_count, valores 10 a 4) midió plano entre 21.32 y 21.54 [M] con un control EXP10 que debía bajar y no bajó: el override nunca aplicó. La hipótesis vigente [D] es de clave o arquitectura: el tag del modelo es qwen3_5_moe, no qwen3moe, y el mecanismo override-kv necesita que la clave exista con el nombre exacto en el GGUF. La vía de archivo salta el intermediario: editar los metadatos del GGUF con gguf-py sobre una COPIA (expert_used_count a 6, 5 y 4) y medir directamente. Si aplica, la dosis-respuesta del 20-vías estimaba +15 a 28% [T]; si el número editado no cambia el t/s, el arch ignora el campo y el enigma se cierra por la vía contraria. Coste: unos 40 minutos de script y medición, con el riesgo de calidad proporcional al recorte de expertos activos.

D4 y D5 — Binarios alternativos: Vulkan e ik_llama.cpp

D4, el zip Vulkan (llama-b11856-bin-win-vulkan-x64.zip, nombre verbatim de los assets oficiales [F]) es el plan B de backend: en NVIDIA suele perder contra CUDA [T], el release b11640 mejoró su scheduling [F], y su rol en el catálogo es doble: rescate documentado (la acceptance 0.89 estable de MTP en Vulkan [D]) y una medición de comparación de backend que cuesta una descarga y una corrida. D5, ik_llama.cpp, es el fork con kernels MoE e IQK que la comunidad reporta hasta ~1.9x en MoE [F]; su gate es duro y sin verificar: soporte del arch qwen3_5_moe con sus capas DeltaNet híbridas. Si el arch no está soportado, la vía muere en el gate; si lo está, es la única vía de motor fuera de mainline del catálogo.

D6 — Caches de slot: TTFT, no decode

El server trae persistencia de KV por slot [F dump]: --slot-save-path (path to save slot kv cache, disabled por defecto) junto a la similitud de prompt -sps (default 0.10). La mecánica comunitaria [F, r/LocalLLaMA] es pre-sembrar los slots con la KV del system prompt (guardar con slots/0?save y restaurar por slot) y ordenar los prompts por prefijo común para maximizar el reuso. No sube el t/s de decode: baja el tiempo hasta la primera respuesta en conversaciones repetitivas, que es el eje de coste por turno del proyecto. Queda excluido de este catálogo, por decisión del Director, todo draft GGUF y EAGLE-3: la especulación hereda el veredicto del protocolo T2.

8. Eje E — Windows: energía, Defender y pagefile

La cuarta familia toca el sistema operativo, no la BIOS (restricción del proyecto). Primera palanca: el plan de energía Alto rendimiento con powercfg /setactive 8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c [F, GUID estándar de Windows documentado por Microsoft y foros], que desactiva el parking de núcleos y evita los estados de reposo del procesador entre pasos del decode. Expectativa de 0 a 5% [T] por estabilización de relojes, dos minutos de ejecución y totalmente reversible con el GUID del plan equilibrado (381b4222-f694-41f0-9685-ff5bb260df2e). Es el equivalente software, y medible, de lo que el eje de energía del informe de 20 vías dejó pendiente con HWiNFO.

Segunda palanca: la exclusión de Defender para las carpetas de trabajo (D:\llama.cpp y la carpeta de modelos), la fase G6 que quedó pendiente del barrido [D]. El mecanismo: los 14.27 GB de carga del modelo, los mapeos de memoria y los logs del server atraviesan el filtro en tiempo real en cada arranque. Expectativa [T]: mejora en load_s y estabilidad del prefill frío, no en el decode steady-state; diez minutos de configuración con la contrapartida de seguridad aceptable de acotar la exclusión a carpetas concretas.

Tercera palanca: pagefile ampliado y de tamaño fijo en D: (cruce con la RUTA 5 del proyecto). Los pageouts son la segunda causa documentada del techo [D] con 14.27 GB de modelo sobre 16 GB de RAM; un pagefile grande no acelera el decode, pero evita los hundimientos cuando la RAM se aprieta, que es exactamente el riesgo de la vía de concurrencia (capítulo 5) y del cualquier experimento que pida RAM extra. Notas finales del eje: start /high es redundante porque --prio 2 ya está en el lanzador [F dump]; las hugepages no tuvieron efecto medible [F, issue #2251]; y el XMP de BIOS queda fuera por la restricción del proyecto, aunque aprendido.md lo marca como la única palanca de coste cero restante [D].

9. Apéndice: flags verificadas del build b11856

Este apéndice es el puente entre el catálogo y el comando. Cada fila lleva la descripción verbatim del dump del build que corre en el rig (llama-flags-nuevo.txt, la fuente de verdad número uno del proyecto) y su estado hoy: ya aplicada en el lanzador, neutral con número medido, veneno o pendiente de medir en uno de los ejes. La jerarquía del proyecto se mantiene: antes de escribir un comando a mano, el dump manda sobre este informe y este informe sobre la memoria. Si el Director actualiza el binario (vía D1), el dump se regenera con PowerShell (llama-server --help redirigido a archivo con encoding unicode) y se sube al repo para re-auditar las columnas de esta tabla.

Flag (dump b11856)	Verbatim / mecanismo	Estado en el rig
-np, --parallel N	number of server slots (default: -1, -1 = auto)	Sin probar: vía del capítulo 5
-cb, --cont-batching	batching continuo (dinámico) del server	Activo por defecto
-bs, --backend-sampling	enable backend sampling (experimental) (default: disabled)	Sin probar: acompaña a -np (cap. 5)
-cram, --cache-ram N	set the maximum cache size in MiB (default: 8192)	Default; sin tocar
-sps, --slot-prompt-similarity	match de prompt por slot (default 0.10; 0.0 = disabled)	Sin probar: cap. 7 (D6)

Flag (dump b11856)	Verbatim / mecanismo	Estado en el rig
--slot-save-path PATH	path to save slot kv cache (default: disabled)	Sin probar: cap. 7 (D6)
-ot, --override-tensor y -otd	patrón de nombre de tensor = tipo de buffer, lista por comas	Colocación fina sin probar: cap. 4
-fit, --fit [on|off]; -fitc N	ajusta los argumentos no fijados para caber en la VRAM del dispositivo	Abortado (bug #27501): retest en D1 (cap. 7)
--moe-cache-mib N	caché GPU de los expertos residentes en RAM host	VENENO medido: 13.51 / 17.22 t/s
--op-offload / --no-op-offload	offload de operaciones de host a dispositivo (default: true)	Apagarlo midió 20.48 (pierde)
GGML_OP_OFFLOAD_MIN_BATCH (env)	umbral de lotes que activa el offload de prompt processing a GPU (default 32)	Sin probar: cap. 6
--prio-batch N (y -draft)	prioridad del hilo de batch: 0-normal, 1-medium, 2-high, 3-realtime	Sin probar: cap. 6
-tb, --threads-batch N	hilos de batch y prompt processing (default: mismos que --threads)	ORO-TB8 neutro: 21.45 t/s
-kvu, --kv-unified; --kv-unified-per-slot	KV unificada entre secuencias (default: enabled) y límite de contexto por slot	Default ON; per-slot útil si entra -np (cap. 5)
-lcs, --lookup-cache-static FNAME	caché estática de lookup para decodificación por lookup	Cerrado: ngram con acceptance <20%
-ctxcp, --ctx-checkpoints N	checkpoints de contexto por slot	Sin necesidad en el uso actual

Tabla 3. Flags del dump b11856 relevantes para el catálogo, con su estado medido o su eje.

Tres ausencias de esta tabla son información tan valiosa como sus filas. El build no lista ninguna variable de entorno CUDA en --help (GGML_OP_OFFLOAD_MIN_BATCH y GGML_CUDA_ENABLE_UNIFIED_MEMORY se documentan en fuentes externas y se fijan en el lanzador con set, no con flags). El par --fit/-fitc aparece ligado al empaquetado automático de VRAM, no a la colocación manual: por eso el eje A usa -ot y el D1 re-testea -fit, sin mezclarlos. Y cualquier flag que no esté aquí y aparezca en un foro debe tratarse como sospechosa hasta grep contra el dump, que es exactamente la regla que fundó flags-verificadas-LAIA.txt.

10. Antivenenos: lo que NO repetir

La mitad del valor de un catálogo está en lo que prohíbe. Esta tabla concentra las palancas ya medidas que perdieron en el rig, con su número y su mecanismo, para que el catálogo de los capítulos 4 a 8 no roce ninguno de estos espacios por accidente. La regla de uso es la de aprendido.md: antes de probar cualquier flag nueva, mirar aquí; lo que tiene veredicto no se re-prueba sin un cambio físico (RAM más rápida, más VRAM, un build que cambie el mecanismo).

Palanca	Número [M]	Por qué pierde
--moe-cache-mib 256 / 512	13.51 / 17.22	La caché roba VRAM a los pesos densos; el driver desborda a RAM compartida por PCIe
-ncmoe con -ngl 99 fijo	5.58 a 7.33	El auto-fit no puede actuar; la VRAM se desborda por la fuerza (VMM)
Afinidad CPU (-Cr, -C, máscaras)	13.37 a 19.48	Enmascarar hilos rompe el planificador de Windows
-t 3 / -t 5	18.29 / 18.75	Núcleo sin uso; hyperthreading peleándose por el núcleo físico
--mlock	prohibido	14.27 GB de modelo en 16 GB de RAM: sin margen para fijar el modelo
MTP (barrido-25)	16.62 a 18.53	El draft paga la misma DDR4; solo re-test con acceptance (protocolo T2)
ngram (acceptance <20%)	18.09 / 20.70	Fichas rechazadas son trabajo tirado
KV q4_0 sola, -c 8192	20.21 / 20.38	Regala calidad a cambio de nada: la KV ya es chica por DeltaNet
--no-op-offload	20.48	El offload de ops a GPU ayuda; apagarlo pierde
REPACKOFF	20.41	El repack de kernels gana; apagarlo pierde
Combos de ganadores	18.98 a 21.33	Las palancas rozan la misma fricción y no suman
GGML_CUDA_ENABLE_UNIFIED_MEMORY	[T]	Paging PCIe del driver: mismo mecanismo del veneno moe-cache; solo como experimento

Tabla 4. Los doce antivenenos del rig, con su número medido o su fundamento.

Cada antiveneno además orienta el catálogo. El veneno de moe-cache explica por qué la vía de concurrencia (capítulo 5) exige medir con contexto 4096 y guardarrrail de VRAM; el veneno de -ncmoe con -ngl fijo explica por qué todo el eje A trabaja con -ngl auto y -fitt; y el veneno de los combos explica por qué este informe es un catálogo de vías independientes y no una receta de super-comando: en este rig, las palancas se validan una por una y solo se apilan después de medir el apilamiento.

11. Matriz maestra y fuentes

La matriz concentra el catálogo completo en una vista: vía, eje, tipo de palanca, evidencia que la sostiene, ganancia esperada y riesgo o coste de medición. Es un catálogo, no un plan: no hay orden de ejecución ni dependencias, tal como pidió el Director. Las ganancias [T] y [D] son hipótesis con física y fuentes detrás; las [M] son mediciones del rig citadas en el capítulo 3.

Vía	Eje	Tipo	Evidencia	Ganancia esperada	Riesgo / coste
-np 2-4 + -cb (+ -bs)	B	Config	[F] #18308 + física MoE [T]	x1.5-2.5 agregado [T]	KV xN en VRAM; latencia por chat; driver de N clientes
Requant IQ3 por tensor + imatrix	D2	Archivo	[F] manpage, #12741, bartowski	+15-30% decode [T]	Calidad (gate chateando); dequant exótico [D]; 1-2 h
Metadatos expert_used_count	D3	Archivo	[M] EXP no-op + [F] tag qwen3_5_moe	+15-28% condicional [T]	Calidad; 40 min con gguf-py sobre copia

Vía	Eje	Tipo	Evidencia	Ganancia esperada	Riesgo / coste
-fit real (binario nuevo)	D1	Binario	[M] FIT-A 22.40 con abort + [F] #27501	+5-8% [T]	Bajo; zip oficial 12.4
GGML_OP_OFFLOAD_MIN_BATCH	C	Entorno	[F] guía HF verbatim	Prefill ±10-30% [D]	Bajo; 3 valores, 20 min
-ub 2048 -b 8192	C	Config	[M] UB512/UB1K	+5-12% prefill [T]	Buffers VRAM (frontera FIT-C)
-ot fino (2-3 capas a GPU)	A	Config	[F] PR #11397 + [M] NCM37C4	+0.3-1.5% [T]; habilitadora	VRAM; solo -ngl auto + -fitt
--prio-batch 2	C	Config	[F] dump L-prio-batch	Pequeño [T]	Ninguno; 12 min
powercfg Alto rendimiento	E	Windows	[F] GUID estándar MS	0-5% [T]	Ninguno; reversible
Exclusión Defender (G6)	E	Windows	[D] pendiente del barrido	load_s y estabilidad [T]	Seguridad: acotar a carpetas
Pagefile fijo en D:	E	Windows	[D] RUTA 5	Estabilidad [T]	Disco
--slot-save-path + -sps	D6	Config	[F] dump + r/LocalLLaMA	TTFT en chats repetidos [T]	Disco para caches
Zip Vulkan	D4	Binario	[F] assets + b11640	Comparación de backend [D]	Probable pérdida vs CUDA [T]
ik_llama.cpp	D5	Binario	[F] ~1.9x MoE comunidad	1.5-2x si soporta [D]	GATE arch qwen3_5_moe sin verificar
Draft GGUF / EAGLE-3 / MTP	excl.	Archivo	Hereda veredicto T2	excluido por decisión	Requiere acceptance del tester-mtp

Tabla 5. Matriz maestra del catálogo. Ganancias [T]/[D] = hipótesis; nada se adopta sin medir.

Las fuentes del catálogo, con las citas que sustentan las etiquetas [F], quedan a continuación. Las fuentes locales del rig (dump de flags b11856, aprendido.md, CSV del barrido-30, logs y RUTAS.md) están en el repositorio del proyecto; las externas se listan con su referencia verificada durante esta investigación.

Dump local del build (fuente de verdad #1): llama-flags-nuevo.txt, llama-server --help de b11856. Verbatim: -np/--parallel (default -1); -bs, --backend-sampling "enable backend sampling (experimental) (default: disabled)"; --op-offload/--no-op-offload (default: true); -cram, --cache-ram N (default: 8192); -sps/--slot-prompt-similarity (default 0.10); -fit/--fit [on|off] y -fitc; -ot/--override-tensor y -otd; --slot-save-path; --prio-batch.

[M] Local: barrido-30-resultados.csv (37 configs, protocolo 4x256) y aprendido.md (veredictos del barrido-25, incluidos moe-cache-mib 256/512 = 13.51/17.22 y la familia -ncmoe con -ngl 99 = 5.58 a 7.33).

[F] llama.cpp PR #11397 "llama: add option to override model tensor buffers": verbatim "to force the experts in MoE models to stay on the CPU, while offloading the rest to the GPU, you could use -ngl 99 -ot exps=CPU".

[F] r/LocalLLaMA "Don't Offload GGUF Layers, Offload Tensors! 200%+ Gen...": patrón -ot ".ffn_.*_exps.=CPU".

[F] HuggingFace blog, Doctor-Shotgun, "Performant local mixture-of-experts CPU inference with llama.cpp" (30-ene-2026): GGML_OP_OFFLOAD_MIN_BATCH verbatim (umbral 32, "Your prompt batch likely needs to be at least several hundred"); patrones -ot por rango de capas.

[F] llama.cpp discussion #18308 "Optimal parameters for parallel inference using llama-server" (5-ene-2026): tabla TG 295.25 (np1) a 1430.11 t/s (np32); "I'm not really getting a significant speedup of -np beyond 4"; ggerganov sobre sampling en CPU y --backend-sampling condicionado a saturación.

[F] Release notes v0.7.0 de llama.cpp: "New GPU cache for host-memory MoE experts (--moe-cache-mib): experts run on the GPU from an LRU cache, up to 2.2x faster decode #29887"; puerto por defecto 9931.

[F] llama.cpp discussion #15650: "Flash Attention is default now (turn it off with -fa off)" — coherente con -fa [on|off|auto] del dump.

[F] Manpage llama-quantize: "an advanced option to selectively quantize tensors"; "--imatrix uses data in file generated by llama-imatrix as importance matrix". Discusión #12741 "Experimenting with custom quants using --tensor-type" (PR #12511).

[F] HuggingFace blog, bartowski, "Per-tensor layout maps for GGUF quantization" (10-sep-2026): sensibilidad de shexp ("pretty sensitive despite being incredibly small") y asimetría atención/expertos.

[F] arXiv 2601.14277, "Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization": coste de calidad de la familia K/IQ en benchmarks y perplexity.

[F] freedom.tech, release history de llama.cpp: b11871 (25-oct-2026), b11747 "Fixes CUDA flash-attention race conditions" (19-oct-2026), b11640 "Vulkan backend gets L2-friendly workgroup scheduling for matrix multiplication" (15-oct-2026).

[F] PR #8035 "ggml-cuda: Adding support for unified memory" (GGML_CUDA_ENABLE_UNIFIED_MEMORY): "useful when the model barely fits in VRAM and inference causes OOM"; discusión #14269 (crecimiento de RAM host con unified memory).

[F] powercfg: GUID del plan Alto rendimiento 8c5e7fda-e8bf-4a96-9a85-a6e23a8c635c (documentación Microsoft y foros de administración); plan equilibrado 381b4222-f694-41f0-9685-ff5bb260df2e.

[F] issue #2251 (hugepages sin efecto) y issue/PR de referencia del abort de --fit (#27501, tensor_buft_overrides already set by user).